SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

Code of Conduct:

I Narravula Navya Sri - 192324220 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*



Signature of the student:

QUESTIONS: **Total Marks: 30**

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.
(a) Analyze the possible reasons for lack of responsiveness in the existing layout.
(b) Design a responsive webpage layout using CSS Flexbox or Grid, ensuring proper alignment of header, navigation, content, and footer.
(c) Demonstrate how the layout adapts to different screen sizes (mobile, tablet, desktop) using media queries.
(d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
(e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Aim

To analyze the causes of a non-responsive retail website and design a responsive webpage using CSS Flexbox, media queries, and appropriate spacing techniques.

(a) Analysis of Reasons for Lack of Responsiveness

A website is considered responsive when its layout automatically adjusts to different screen sizes, including mobile phones, tablets, and desktop computers.

The possible reasons for a lack of responsiveness are:

1. **Fixed-width layouts:** Using fixed pixel values for widths prevents content from adapting to smaller screens.
2. **Missing media queries:** Without media queries, the layout cannot apply different styles for different screen sizes.
3. **Improper use of positioning:** Excessive use of absolute positioning can cause overlapping when the viewport changes.
4. **Inappropriate image sizing:** Images with fixed dimensions may overflow their containers.
5. **Incorrect spacing:** Excessive margins and padding can cause alignment problems on smaller devices.
6. **Lack of flexible layouts:** Not using Flexbox or Grid makes it difficult to distribute elements dynamically.
7. **Missing viewport configuration:** Without the viewport meta tag, mobile browsers may render the page at an unexpected width.

(b), (c), and (d) Design and Implementation

The following HTML and CSS create a responsive retail webpage using Flexbox and media queries.

HTML Code:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport"

        content="width=device-width, initial-scale=1.0">

    <title>Responsive Retail Website</title>

    <link rel="stylesheet" href="style.css">

</head>

<body>

    <header class="header">

        <h1>ShopEase</h1>

        <p>Your Everyday Shopping Destination</p>

    </header>

    <nav class="navbar">

        <a href="#">Home</a>

        <a href="#">Products</a>

        <a href="#">Offers</a>

        <a href="#">Contact</a>

    </nav>

    <main class="container">

        <section class="content">

            <h2>Welcome to ShopEase</h2>

            <p>

                Discover quality products at affordable prices.

                Explore our latest collections and special offers.

            </p>
```

```
        </section>
        <aside class="sidebar">
            <h3>Special Offers</h3>
            <p>Get exciting discounts on selected products!</p>
        </aside>
    </main>
    <footer class="footer">
        <p>&copy; 2026 ShopEase. All rights reserved.</p>
    </footer>
</body>
</html>
```

CSS Code (style.css):

```
/* Apply consistent box sizing */
* {
    box-sizing: border-box;
}
body {
    margin: 0;
    font-family: Arial, sans-serif;
    line-height: 1.6;
    color: #333;
}
/* Header */
.header {
    background-color: #2563eb;
    color: white;
    text-align: center;
    padding: 24px 16px;
}
/* Navigation */
```

```
.navbar {
    display: flex;
    justify-content: center;
    align-items: center;
    flex-wrap: wrap;
    gap: 20px;
    background-color: #1e293b;
    padding: 12px 16px;
}
.navbar a {
    color: white;
    text-decoration: none;
    padding: 8px 12px;
}
.navbar a:hover {
    background-color: #475569;
    border-radius: 4px;
}
/* Main content */
.container {
    display: flex;
    align-items: stretch;
    gap: 24px;
    max-width: 1200px;
    margin: 0 auto;
    padding: 24px;
}
.content {
    flex: 2;
    min-width: 0;
```

```
    padding: 24px;
    background-color: #f1f5f9;
    border-radius: 8px;
}
.sidebar {
    flex: 1;
    min-width: 0;
    padding: 24px;
    background-color: #dbeafe;
    border-radius: 8px;
}
/* Footer */
.footer {
    background-color: #1e293b;
    color: white;
    text-align: center;
    padding: 16px;
    margin-top: 16px;
}
/* Mobile devices */
@media (max-width: 600px) {
    .header {
        padding: 16px 12px;
    }
    .navbar {
        flex-direction: column;
        gap: 4px;
    }
    .navbar a {
        width: 100%;
```

```
        text-align: center;
    }
    .container {
        flex-direction: column;
        gap: 16px;
        padding: 16px;
    }
    .content,
    .sidebar {
        width: 100%;
        padding: 16px;
    }
}
/* Tablets */
@media (min-width: 601px) and (max-width: 900px) {
    .container {
        gap: 16px;
        padding: 20px;
    }
    .content {
        flex: 3;
    }
    .sidebar {
        flex: 2;
    }
    .navbar {
        gap: 12px;
    }
}
```

Explanation of Responsive Techniques

1. Flexbox

The display: flex property arranges navigation links and content sections efficiently.

- justify-content: center centers navigation links horizontally.
- align-items: center aligns items along the cross axis.
- flex-wrap: wrap allows navigation items to move to another line when necessary.
- flex-direction: column stacks content vertically on mobile devices.
- gap provides consistent spacing between elements.

2. Media Queries

Media queries apply different CSS rules depending on the viewport width.

Screen size	Width	Layout behavior
Mobile	Up to 600px	Navigation and content stack vertically.
Tablet	601px–900px	Content and sidebar appear side by side with adjusted proportions.
Desktop	Above 900px	Content and sidebar use a wider horizontal layout.

3. Spacing and Alignment

- box-sizing: border-box includes padding and borders in the element's declared width.
- max-width: 1200px prevents the content from becoming excessively wide.
- margin: 0 auto horizontally centers the main container.
- gap creates consistent spacing between sections.
- padding provides space between content and element boundaries.
- min-width: 0 allows Flexbox children to shrink appropriately.

(e) Evaluation and Further Improvements

The responsive design provides the following benefits:

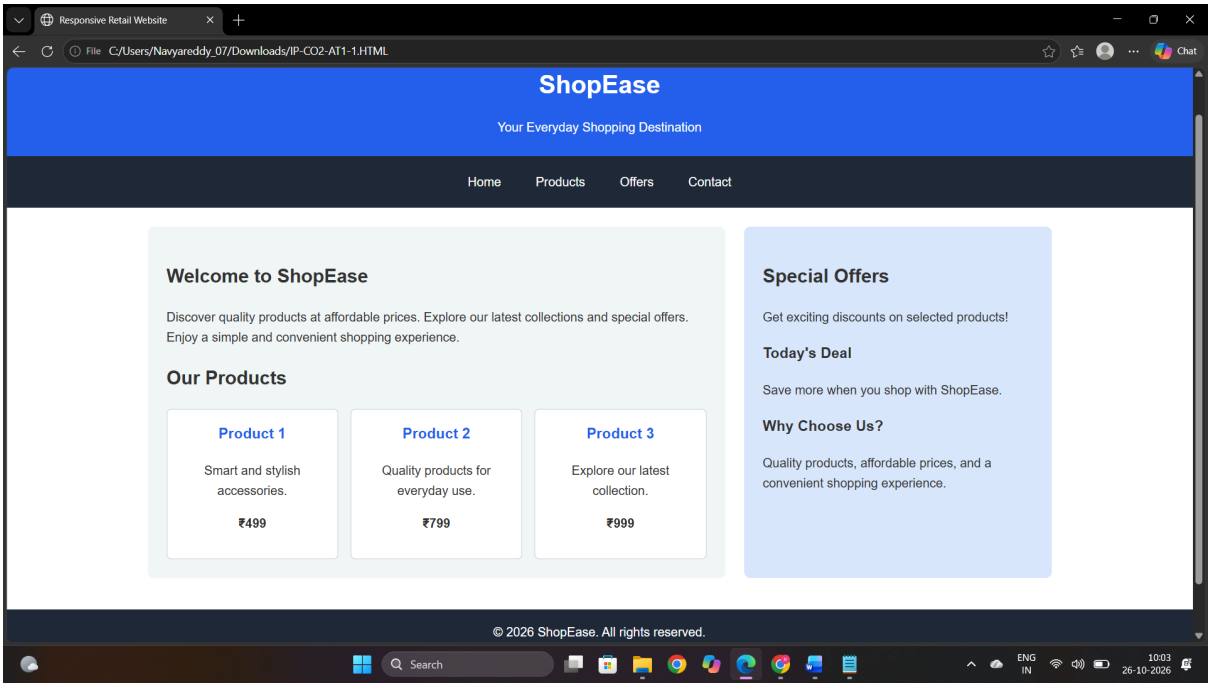
1. Content adapts to different screen sizes.
2. Navigation remains readable on mobile devices.
3. Flexbox distributes available space efficiently.
4. Consistent spacing improves visual appearance.
5. The layout reduces the likelihood of horizontal overflow.

Further improvements:

- Use responsive images with max-width: 100%; height: auto.
- Add accessible navigation controls for smaller screens if necessary.

- Test the page on multiple browsers and devices.
- Use CSS Grid for more complex product galleries.
- Check text readability, keyboard navigation, and color contrast.

Result: A responsive retail webpage was designed using CSS Flexbox, media queries, and consistent spacing techniques.



Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.
(a) Analyze the issues in the existing form validation mechanism.
(b) Design input fields for email and phone number with appropriate HTML attributes.
(c) Implement JavaScript validation using Regular Expressions for both email and phone number.
(d) Develop a mechanism to display real-time error messages for invalid inputs.
(e) Evaluate the validation system and suggest enhancements for better user experience and security.

Aim

To develop a registration form that validates email addresses and phone numbers using HTML attributes, JavaScript, and regular expressions, with real-time error messages.

(a) Analysis of Existing Validation Issues

The existing registration form may have the following problems:

1. It accepts invalid email addresses because validation rules are missing or incomplete.
2. It allows letters or special characters in phone number fields.
3. It does not check whether required fields are empty.
4. It displays no immediate feedback when invalid information is entered.
5. It may rely only on HTML validation without additional JavaScript checks.
6. It may trust client-side validation without validating submitted data on the server.

These issues can result in incorrect records, failed communication, and poor user experience.

(b), (c), and (d) Form Design and Validation

The following implementation uses HTML attributes and JavaScript regular expressions to validate email and phone number inputs. Error messages appear as the user types.

Complete HTML and JavaScript Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
      content="width=device-width, initial-scale=1.0">
  <title>Registration Form</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f1f5f9;
      margin: 0;
      padding: 20px;
    }
    .form-container {
      max-width: 400px;
      margin: 30px auto;
      padding: 24px;
      background: white;
      border-radius: 8px;
      box-shadow: 0 2px 10px #00000015;
    }
    label {
      display: block;
      margin-top: 16px;
      font-weight: bold;
    }
    input {
      width: 100%;
      box-sizing: border-box;
      padding: 10px;
      margin-top: 6px;
      border: 1px solid #94a3b8;
      border-radius: 4px;
    }
    input:focus {
      outline: 2px solid #2563eb;
```

```
            outline-offset: 1px;
        }
        input.invalid {
            border-color: #dc2626;
        }
        .error {
            color: #dc2626;
            font-size: 14px;
            margin-top: 4px;
        }
        .success {
            color: #15803d;
            margin-top: 12px;
        }
        button {
            width: 100%;
            padding: 12px;
            margin-top: 20px;
            background: #2563eb;
            color: white;
            border: none;
            border-radius: 4px;
            cursor: pointer;
        }
        button:hover {
            background: #1d4ed8;
        }
    </style>
</head>
<body>
    <div class="form-container">
        <h2>Registration Form</h2>
        <form id="registrationForm" novalidate>
            <label for="email">Email Address</label>
            <input
                type="email"
                id="email"
                name="email"
                placeholder="example@email.com"
                autocomplete="email"
                required
                aria-describedby="emailError">
            <div id="emailError"
                 class="error"
                 aria-live="polite"></div>
            <label for="phone">Phone Number</label>
            <input
                type="tel"
                id="phone"
                name="phone"
                placeholder="9876543210"
                autocomplete="tel"
                inputmode="tel"
                required
                aria-describedby="phoneError">
            <div id="phoneError"
                 class="error"
                 aria-live="polite"></div>
            <button type="submit">Register</button>
            <p id="formMessage"
```

```
            class="success"
            aria-live="polite"></p>
        </form>
    </div>
    <script>
        const form = document.getElementById("registrationForm");
        const email = document.getElementById("email");
        const phone = document.getElementById("phone");
        const emailError = document.getElementById("emailError");
        const phoneError = document.getElementById("phoneError");
        const formMessage = document.getElementById("formMessage");
        // Basic email format validation
        const emailRegex =
            /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        // Ten-digit Indian mobile number beginning with 6, 7, 8, or 9
        const phoneRegex = /^[6-9]\d{9}$/;
        function validateEmail() {
            const value = email.value.trim();
            if (value === "") {
                emailError.textContent = "Email is required.";
            } else if (!emailRegex.test(value)) {
                emailError.textContent =
                    "Enter a valid email address.";
            } else {
                emailError.textContent = "";
            }
            email.classList.toggle(
                "invalid",
                emailError.textContent !== ""
            );
            return emailError.textContent === "";
        }
        function validatePhone() {
            const value = phone.value.trim();
            if (value === "") {
                phoneError.textContent =
                    "Phone number is required.";
            } else if (!phoneRegex.test(value)) {
                phoneError.textContent =
                    "Enter a valid 10-digit Indian mobile number.";
            } else {
                phoneError.textContent = "";
            }
            phone.classList.toggle(
                "invalid",
                phoneError.textContent !== ""
            );
            return phoneError.textContent === "";
        }
        // Validate while typing and when leaving a field
        email.addEventListener("input", () => {
            validateEmail();
            formMessage.textContent = "";
        });

        email.addEventListener("blur", validateEmail);
        phone.addEventListener("input", () => {
            validatePhone();
            formMessage.textContent = "";
        });
```

```
        phone.addEventListener("blur", validatePhone);
        // Validate again before submission
        form.addEventListener("submit", function(event) {
            event.preventDefault();
            const validEmail = validateEmail();
            const validPhone = validatePhone();
            if (validEmail && validPhone) {
                formMessage.textContent =
                    "Validation successful! Form is ready to submit.";
            } else {
                formMessage.textContent = "";
            }
        });
    </script>
</body>
</html>
```

Explanation of the Implementation

1. HTML Input Attributes

Attribute	Purpose
type="email"	Provides email-specific input behavior and browser validation.
type="tel"	Identifies the field as a telephone number.
required	Marks the input as mandatory.
autocomplete	Helps browsers autofill known user information.
inputmode="tel"	Suggests a telephone-friendly mobile keyboard.
aria-describedby	Associates each input with its error message.

The form uses novalidate so that JavaScript can display its own error messages instead of relying on the browser's default submission messages.

2. Regular Expressions

Email validation:

```
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

This checks for:

- At least one character before @.
- A domain portion after @.
- A dot followed by at least one character.
- No whitespace.

It is a basic format check, not a complete implementation of all valid email-address rules.

Phone validation:

```
const phoneRegex = /^[6-9]\d{9}$/;
```

This checks for:

- Exactly ten digits.
- A first digit between 6 and 9.
- No spaces, country code, or other characters.

This particular pattern is intended for ten-digit Indian mobile numbers. Other countries require different validation rules.

3. Real-Time Validation

The input event runs whenever the user changes a field. The blur event validates the field when the user leaves it.

The validateEmail() and validatePhone() functions check the input and display an appropriate error message.

The classList.toggle() method applies a red border when the input is invalid.

4. Form Submission

When the user submits the form:

1. JavaScript validates the email.
2. JavaScript validates the phone number.
3. If both fields are valid, a success message appears.
4. If either field is invalid, the relevant error message remains visible.

The example demonstrates validation only; it does not transmit or store registration data.

(e) Evaluation and Enhancements

Advantages:

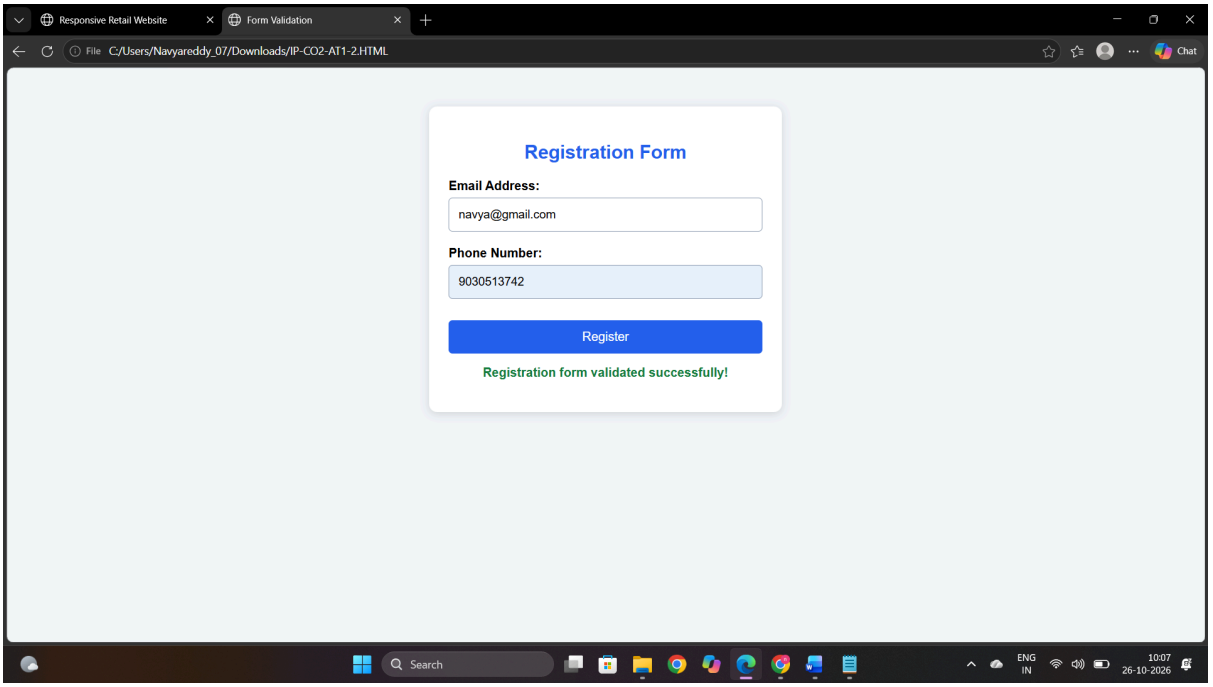
- Invalid inputs are identified immediately.
- Error messages explain what needs to be corrected.
- Regular expressions enforce basic format rules.
- HTML attributes improve accessibility and mobile input behavior.
- The form avoids indicating success when validation fails.

Further improvements:

1. Validate all fields on the server before accepting data.
2. Normalize phone numbers and support country codes where required.
3. Use a more comprehensive email-validation library if strict address validation is necessary.
4. Provide clear labels and keyboard-accessible error messages.
5. Avoid displaying errors before the user has had an opportunity to enter information.
6. Add rate limiting, CSRF protection where applicable, and secure server-side processing.
7. Never treat a syntactically valid email or phone number as proof that it belongs to the user; use verification when necessary.

Security note: JavaScript validation improves usability but can be bypassed. Server-side validation is essential whenever data is submitted to a backend.

Result: A registration form was developed with real-time email and phone validation using JavaScript regular expressions.



Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.
(a) Analyze the CSS properties responsible for layout overlapping issues.
(b) Examine the role of the CSS box model (margin, border, padding, content) in layout design.
(c) Debug the layout using appropriate positioning techniques (relative, absolute, flex, grid).
(d) Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
(e) Evaluate the corrected layout and suggest best practices for maintaining consistency.

Aim

To identify and correct CSS layout overlapping issues using the CSS box model, positioning techniques, Flexbox, and Grid.

(a) Analysis of CSS Properties Responsible for Overlapping

Layout overlapping may occur due to the following CSS properties and practices:

1. **Absolute positioning:** Elements positioned relative to a container may overlap other content.
2. **Fixed dimensions:** Elements with fixed widths and heights may not fit smaller viewports.
3. **Negative margins:** Negative margins can pull elements into neighboring sections.
4. **Excessive padding and borders:** These may cause elements to exceed their expected widths when the default box model is used.
5. **Incorrect z-index:** Positioned elements with higher stacking levels can cover other elements.
6. **Floats:** Floated elements can cause surrounding content to wrap unexpectedly if the layout is not cleared properly.
7. **Improper Flexbox settings:** Flex items may overflow if they cannot shrink or if their minimum sizes are too large.
8. **Improper Grid sizing:** Fixed track sizes may cause content to overflow the available space.

(b) Role of the CSS Box Model

The CSS box model describes the space occupied by an element.

It consists of four components:

1. **Content:** The text, image, or other material inside an element.
2. **Padding:** The space between the content and the border.
3. **Border:** The line surrounding the padding and content.
4. **Margin:** The space outside the border, separating the element from other elements.

For an element using the default content-box model:

Total width = Content width + Left and right padding + Left and right borders + Left and right margins

For example, consider an element with:

- Content width = 300px
- Left and right padding = 20px each
- Left and right borders = 2px each
- Left and right margins = 10px each

Its total horizontal space is:

300 + 40 + 4 + 20 = **364px**

If the available container width is only 340px, the element may cause horizontal overflow.

Using box-sizing: border-box makes the declared width include the content, padding, and borders, which simplifies layout calculations. Margins remain outside the declared width.

(c) Debugging the Layout

Consider the following problematic CSS:

```
.container {
   width: 1000px;
}
.box {
   position: absolute;
   width: 500px;
   height: 300px;
}
.sidebar {
   position: absolute;
   left: 450px;
   width: 500px;
}
```

Problems:

- The container has a fixed width.
- Both sections use absolute positioning.
- The sidebar overlaps the first box because their horizontal positions intersect.
- The fixed dimensions do not adapt to smaller viewports.

The layout can be improved by replacing absolute positioning with Flexbox or Grid for the main content.

(d) Corrected CSS Implementation

The following example uses CSS Grid to create two content sections that automatically adjust to the available width.

HTML Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
   <title>CSS Layout Debugging</title>
   <link rel="stylesheet" href="layout.css">
</head>
<body>
   <header class="header">
      <h1>Layout Debugging Example</h1>
   </header>
   <main class="container">
      <section class="main-content">
         <h2>Main Content</h2>
         <p>
            This section contains the primary webpage content.
            It adjusts its size according to the available space.
         </p>
      </section>
```

```
        <aside class="sidebar">
            <h2>Sidebar</h2>
            <p>
                This section contains additional information
                and remains beside the main content on wider screens.
            </p>
        </aside>
    </main>
    <footer class="footer">
        <p>Responsive layout with CSS Grid</p>
    </footer>
</body>
</html>
```

Corrected CSS Code (layout.css):

```
/* Use a predictable box model */

* {

    box-sizing: border-box;

}

body {

    margin: 0;

    font-family: Arial, sans-serif;

    line-height: 1.6;

    color: #1f2937;

}

/* Header */

.header {

    background-color: #1d4ed8;

    color: white;

    text-align: center;

    padding: 20px;

}

/* Responsive grid layout */

.container {

    display: grid;

    grid-template-columns: minmax(0, 2fr) minmax(0, 1fr);

    gap: 20px;
```

```
    width: 100%;
    max-width: 1200px;
    margin: 0 auto;
    padding: 24px;
}
.main-content,
.sidebar {
    min-width: 0;
    padding: 20px;
    border-radius: 8px;
    overflow-wrap: anywhere;
}
.main-content {
    background-color: #e0f2fe;
}
.sidebar {
    background-color: #fef3c7;
}
/* Footer */
.footer {
    background-color: #1f2937;
    color: white;
    text-align: center;
    padding: 16px;
}
/* Tablet and mobile layout */
@media (max-width: 768px) {
    .container {
        grid-template-columns: minmax(0, 1fr);
        gap: 16px;
```

```
        padding: 16px;
    }
    .main-content,
    .sidebar {
        padding: 16px;
    }
}
/* Additional adjustments for narrow mobile screens */
@media (max-width: 480px) {
    .header {
        padding: 14px 10px;
    }
    .header h1 {
        font-size: 22px;
    }
    .container {
        padding: 12px;
        gap: 12px;
    }
    .main-content,
    .sidebar {
        padding: 12px;
    }
}
```

Explanation of the Corrected Layout

1. CSS Grid

```
grid-template-columns: minmax(0, 2fr) minmax(0, 1fr);
```

This creates two columns, with the main content receiving approximately twice the available track space of the sidebar.

The minmax(0, ...) expressions allow the grid tracks to shrink below their intrinsic content sizes when necessary, reducing overflow.

2. Responsive Media Queries

At widths of 768px or less:

```
grid-template-columns: minmax(0, 1fr);
```

The main content and sidebar are placed in a single column, preventing horizontal crowding.

At widths of 480px or less, the layout further reduces padding and heading size to fit narrow screens.

3. Box-Sizing Correction

```
* {
  box-sizing: border-box;
}
```

This ensures that padding and borders are included in the declared width and height of elements.

4. Content Wrapping

```
overflow-wrap: anywhere;
```

This allows long strings, such as unusually long URLs, to wrap instead of forcing the content area to expand horizontally.

5. Centered Container

```
max-width: 1200px;
margin: 0 auto;
```

The container remains within a reasonable maximum width and is centered on larger screens.

Role of Positioning Techniques

Technique	Purpose	Recommended use
position: relative	Positions an element relative to its normal position and establishes a reference for positioned descendants.	Small visual adjustments and positioning anchors.

position: absolute	Removes an element from normal document flow.	Badges, icons, overlays, and decorative elements.
Flexbox	Arranges elements along one primary axis.	Navigation bars, toolbars, and aligned components.
Grid	Arranges content in rows and columns.	Page layouts, dashboards, and sidebars.
Normal document flow	Places elements according to the document structure.	Most text and content sections.

For ordinary page content, Flexbox or Grid is usually more reliable than absolute positioning because the layout can adapt to the content's size.

(e) Evaluation and Best Practices

Evaluation:

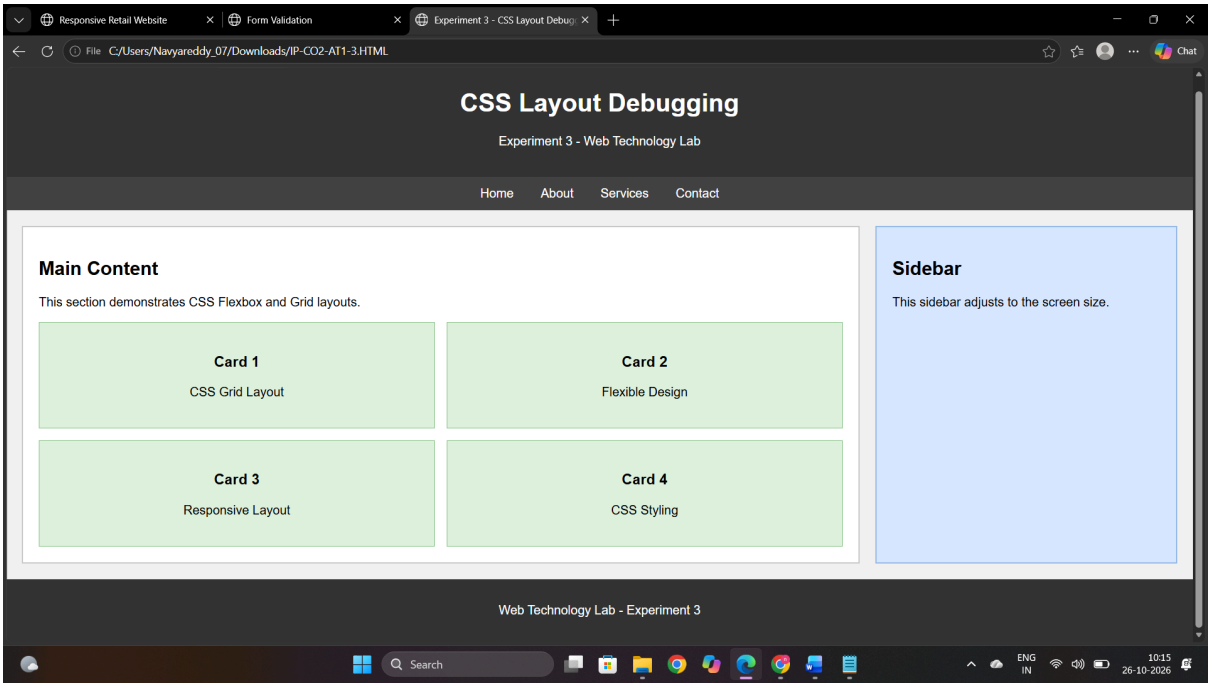
The corrected layout offers the following improvements:

- The main content and sidebar no longer depend on fixed horizontal positions.
- The grid adjusts automatically to different viewport widths.
- Padding and borders are included in width calculations.
- Content can wrap instead of forcing horizontal overflow.
- Mobile devices display the content in a single column.
- The maximum container width improves readability on large screens.

Best practices:

1. Use Flexbox for one-dimensional layouts and Grid for two-dimensional layouts.
2. Avoid fixed widths for primary page containers.
3. Apply box-sizing: border-box consistently.
4. Use relative units, flexible tracks, and max-width where appropriate.
5. Reserve absolute positioning for elements that genuinely need to be removed from normal document flow.
6. Inspect layouts at multiple viewport widths using browser developer tools.
7. Check for long text, large images, and other content that might cause overflow.
8. Test keyboard accessibility and zoom behavior.
9. Avoid unnecessary z-index values and negative margins.
10. Test the page across browsers and devices.

Result: The overlapping layout was corrected using CSS Grid, responsive media queries, and box-model adjustments, improving alignment and preventing common resizing problems.



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation